

```

0000 compiler-- Copyright (C) 2024-2026 Vincent MORIN - Universit  de Bretagne Occidentale-- This program is free software: you can redistribute it and/or
0001 or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or
0002 (at your option) any later version. This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even
0003 the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details. You should have received a copy of the GNU General Public License along with this program. If not, see https://www.gnu.org/licenses/.
0004
0005 -- Vincent MORIN-Universit  de Bretagne Occidentale-janvier 2025
0006
0007 --package body--PRINT_NOD--is
0008 --1-2-3-4-5-6-7-8-9-A-B-C-with UNCHECKED CONVERSION;separate(-IDL)
0009 EGER; DUMMY:= array (1..4) of CHARACTER; := (CHARACTER'VAL (1), ASCII.NUL, ASCII.NUL, ASCII.NUL); for DUMMY use at DUMMY2'ADDRESS;
0010 IS_LITTLE_ENDIAN:= constant BOOLEAN:= (DUMMY2 = 1);
0011 --FUNCTION L_PNN--function L_PNN ( NN : NODE_NAME ) return NATURAL is STR:= constant STRING:= NODE_NAME'IMAGE(-NN); begin PUT ( STR
0012 R ); return STR'LENGTH; end;
0013 --FUNCTION L_PINT--function L_PINT ( I : INTEGER ) return NATURAL is begin if I < 0 then ENTIER NEGATIF; ENTIER NEGATIF;
0014 nt I < -32767 then PAS DE VALEUR POSITIVE CORRESPONDANTE; PUT (-32768); ECRIRE LA VALEUR NEGATIVE-MINIMALE; return
0015 urn 6; 6 CARACTERES DE LONG; CAS STANDARD DES NEGATIFS-POUR LESQUELS UNE VALEUR POSITIVE CORRESPONDANTE EXISTE;
0016 PUT (-I); METTRE LE SIGNE; return L_PINT(-I) + 1; IMPRIMER LE POSITIF ET RETOURNER LA LONGUEUR (AVEC SIGNE)
0017 ER LE DIV-SUIVI DU MOD EN RETOURNANT LEUR LONGUEUR TOTALE; else POSITIF A 1 SEUL CHIFFRE; PUT (-CHARACTER'VAL ( CHARACTER'POS( '0
0018 ) + I )); IMPRIMER LE CHIFFRE; return 1; end if; end;
0019 --FUNCTION L_PINT--function L_PINT ( P : VPG_IDX ) return NATURAL is begin return L_PINT( INTEGER(P) ); end; function L
0020 PINT ( L : LINE_IDX ) return NATURAL is begin return L_PINT( INTEGER( L ) ); end; function L_PINT ( S : POSITIVE_SHORT ) return NATURAL is begin retur
0021 n L_PINT( INTEGER( S ) ); end; function L_PINT ( C : SRC_COL_IDX ) return NATURAL is begin return L_PINT( INTEGER( C ) ); end;
0022 --FUNCTION PRINT_ABS_TREE--function PRINT_ABS_TREE ( T : TREE ) re
0023 turn INTEGER is IMPRESSION D'UN-POINTEUR DENOTANT UNE ANOMALIE; SIZE:= INTEGER; LE "I?" DE DEBUT PLUS DEUX E ET LE
0024 <?!"-DE FIN" begin PUT ( "I?" ); case T.PT is when P | L => SIZE := SIZE + L_PNN ( T.TY ); PUT ( 'P' ); SIZE := SIZE + L_PINT( T.PG ); PU
0025 T ( 'L' ); SIZE := SIZE + L_PINT( T.LN ); when HI => SIZE := SIZE + L_PNN( T.NOTY ); PUT ( 'S' ); SIZE := SIZE + L_PINT( T.NSIZ ); PUT ( 'A' )
0026 ; SIZE := SIZE + L_PINT( T.ABSS ); when S => SIZE := SIZE + L_PINT( T.COL ); PUT ( 'P' ); SIZE := SIZE + L_PINT( T.SPG ); PUT ( 'L' ); SIZE :=
0027 SIZE + L_PINT( T.SLN ); end case; PUT ( "<?" ); return SIZE; end;
0028 --PROCEDURE PUT_LONG_DGIT--procedure PUT_LONG_DGIT ( I : INTEGER ) is ENTIER A 4 CHIFFRES AVEC ZEROS NON SIGN
0029 IFICATIFS; DUMMY: INTEGER; begin if I < 1000 then PUT ( '0' ); if I < 100 then PUT ( '0' ); if I < 10 then PUT ( '0' );
0030 end if; end if; DUMMY := L_PINT(-I); end; procedure PRINT_TREE( T : TREE ) is DUMMY:=
0031 INTEGER; begin DUMMY := L_PINT_TREE( T ); end-PRINT_TREE; function L_PINT_TREE( T : TREE ) return NATURAL is
0032 function TRAITE_SOURCELINE--return NATURAL is CAS DU NOEUD S (SOURCE_LINE) begin if T.SPG > P
0033 AGE_MAN.HIGH_VPG then LE CHAMP PG EST-HORS DU FICHIER ARBRE (ANORMAL) return PRINT_ABS_TREE( T ); IMPRIMER COMME <?!" end
0034 if; declare NB_CARS: NATURAL; SLPTR: TREE:= ( P, TY=>DN_SOURCELINE, PG=>T.SPG, LN=>T.SLN ); FABRIQUER UN POINTEUR DE SOURCE
0035 LINE-NORMAL AVEC LE POINTEUR SOURCE_POSITION; SLNOD:= D( XD_NUMBER, SLPTR ); XD_NUMBER UN ENTIER begin NB_CARS := L_PRI
0036 NT_TREE( SLPTR ); FAIRE-IMPRIMER LE NOEUD LIGNE SOURCE (PG=LIGNE ET LN=COL) PUT ( 'C' ); ON SUIT AVEC LE-CONTENU if S
0037 LNOD.NOTY:= DN_NUM_VAL then UNE VALEUR NUMERIQUE ENTIERE (16 BITS) NB_CARS := NB_CARS + L_PINT( SLNOD.ABSS ); FAIRE-IMPRIMER
0038 LA VALEUR 16 BITS (NUM_LIGNE) else ANOMALIE NB_CARS := NB_CARS + PRINT_ABS_TREE( SLNOD ); AU FORMAT $$$TY.PG.LN$S PO
0039 UR INFORMATION end if; PUT ( ' ' ); NB_CARS := NB_CARS + L_PINT( T.COL ); PUT ( ' ' ); return NB_CARS + 3;
0040 NOMBRES PLUS (,); end if; end-TRAITE_SOURCELINE;
0041 --FUNCTION TRAITE_NUM_VAL--function TRAITE_NUM_VAL return NATURAL is begin if T.PT = HI then VALEUR
0042 COURTE 16 BITS; if T.NSIZ = 1-then VALEUR NEGATIVE; return L_PINT( INTEGER(-T.ABSS + 1) ); IMPRIMER else
0043 VALEUR POSITIVE; PUT ( '+' ); PREFIXER PAR LE-SIGNE; return L_PINT( T.ABSS + 1 ); IMPRIMER ET DONNER LA TAI
0044 LLE AVEC LE SIGNE EN-PLUS; end if; else if T.PT = P then POINTEUR VERS BLOC GRAND ENTIER; if not ( T.PG in 1.. PAGE_MAN.HIGH_VP
0045 G ) then ANOMALIE SUR L ADRESSE DE-PAGE; return PRINT_ABS_TREE( T ); end if; declare UN_VRAI DN_NUM_VAL
0046 ENTETE:= TREE:= DABS( 0, T ); ENTETE CONTENANT LE NOMBRE DE DGITS; type DOUBLET is array(1..2) of SHORT; function TO_DO
0047 BLET:= new UNCHECKED_080N( TREE, DOUBLET ); DD:= DOUBLET:= TO_DOUBLET( DABS( 1, T ) ); PREMIERE PAIRE DE DGITS BASE 10000
0048
0049 NB_CARS:= INTEGER:= 0; begin--IF DD(1) >= 10000 THEN--10000-EST AJOUTE AU PREMIER POUR NB-NEGATIF if ENTETE.ABSS = 1
0050 then--ABSS 1 POUR NB NEGATIF-PUT ( '-' ); CHIFFRE NEGATIF; else-PUT ( '+' ); DE 0 A 9 999 : CHIFFRE POSITIF
0051 end if; for I in 1.. ENTETE.NSIZ loop--DD := TO_DOUBLET( DABS( I, T ) ); PAIRE-DE DGITS-PUT_LONG_DGIT( INTEGER( DD(1) mod 10
0052 000 ) ); SECOND DIGIT 10 000 AIRE-PUT ( ' ' ); PUT_LONG_DGIT( INTEGER( DD(2) mod 10 000 ) ); PREMIER DIGIT 10 000 AIRE-(MOD POUR-LE
0053 PREMIER if I /= ENTETE.NSIZ-then if (I mod 8) = 1 then--NEW_LINE; end if; PUT ( ' ' ); end if; NB_CARS := NB_CARS + 10;
0054 end loop; return NB_CARS; end; end if; return 0; end; begin case T.PT is when S => return TRAITE_SOURCELINE; w
0055 hen HI => return REPRESENTANT UN ENTIER 16 BITS-STRICTEMENT NEGATIF-OU UNE POSITION SOURCE; if T.NOTY = DN_NUM_VAL then VALE
0056 UR ENTIERE COURTE; return TRAITE_NUM_VAL; else declare NAM:= constant STRING:= NODE_NAME'IMAGE(-T.NOTY ); CHAINE DU TY
0057 E DE NOEUD; begin PUT ( 'C' & NAM & ' ' ); IMPRIMER LE NOM-DU TYPE DE NOEUD; return NAM'LENGTH + 2; AJOUTER SA
0058 LONGUEUR; end if; when P => POINTEUR; if T.TY = DN_NUM_VAL then return TRAITE_NUM_VAL; end if; declare N
0059 B_CARS: INTEGER; begin--PAS UNE VALEUR ENTIERE; declare NAM:= constant STRING:= NODE_NAME'IMAGE(-T.TY ); CHAINE
0060 DU TYPE DE NOEUD; begin PUT ( 'C' & NAM ); IMPRIMER LE NOM-DU TYPE DE NOEUD; NB_CARS := NAM'LENGTH + 1; AJOUTE
0061 R SA LONGUEUR; end; PUT ( ' ' ); NB_CARS := NB_CARS + L_PINT( T.PG ); PAGE OU BLOC DU-POINTEUR; PUT ( 'L' ); NB_
0062 CARS := NB_CARS + L_PINT( T.LN ) + 5; LIGNE-DU POINTEUR ET AJOUTER LES TAILLES; PUT ( ' ' ); if T.PG = 0-then--CAS INHAB
0063 ITUEL AU RELOC; return NB_CARS; ARRETER ICI; end if; if T.TY = DN_TXTREP then CAS PARTICULIER-D'UN TXTREP;
0064 PUT ( ' ' ); if T.PG > PAGE_MAN.HIGH_VPG then--HORS BORNES (1-)?PUT ( "??XT??" ); NB_CARS := NB_CARS + 7; TAILLE N
0065 OM PLUS-UN ET SIX -return NB_CARS; end if; declare NAM:= constant STRING:= PRINT_NAME( T ); begin-PUT( NAM ); NB_CA
0066 RS := NB_CARS + 1 + NAM'LENGTH; end; end if; return NB_CARS; end; when L => null; end case; return 0; end-L_PINT_TR
0067 EE;
0068 --PROCEDURE PRINT_NODE--procedure PRINT_NODE ( T : TREE; INDENT : NATURAL := 0 ) is
0069 --PROCEDURE PRINT_SUB_TREE--procedure PRINT_SUB_TREE ( T : TREE; IND : NATURAL is
0070 A_SIZ:= ATTR_NBR:= N_SPEC( T.TY ).NS_SIZE; N_SIZ:= ATTR_NBR:= A_SIZ; A_SUB:= INTEGER:= N_SPEC( T.TY ).NS_FIRST; TR:= TREE;
0071 SQ:= SEQ_TYPE;
0072 -- procedure PRINT_SUB_TREE ( T : TREE ) is begin PRINT_TREE( T ); if T.PT = P and then ( T.TY = DN_SYMBOL_REP and T.PG > 0 ) t
0073 hen--PUT( PRINT_NAME( T ) ); end if; end PRINT_SUB_TREE; begin if T.TY = DN_HASH then TR := DABS( 0, T ); N_SIZ :=
0074 Y DN_HASH then PUT ( ' ' ); NB_CARS := NB_CARS + L_PINT( T.PG ); PAGE OU BLOC DU-POINTEUR; PUT ( 'L' ); NB_
0075 CARS := NB_CARS + L_PINT( T.LN ) + 5; LIGNE-DU POINTEUR ET AJOUTER LES TAILLES; PUT ( ' ' ); if T.PG = 0-then--CAS INHAB
0076 ITUEL AU RELOC; return NB_CARS; ARRETER ICI; end if; if T.TY = DN_TXTREP then CAS PARTICULIER-D'UN TXTREP;
0077 PUT ( ' ' ); if T.PG > PAGE_MAN.HIGH_VPG then--HORS BORNES (1-)?PUT ( "??XT??" ); NB_CARS := NB_CARS + 7; TAILLE N
0078 OM PLUS-UN ET SIX -return NB_CARS; end if; declare NAM:= constant STRING:= PRINT_NAME( T ); begin-PUT( NAM ); NB_CA
0079 RS := NB_CARS + 1 + NAM'LENGTH; end; end if; return NB_CARS; end; when L => null; end case; return 0; end-L_PINT_TR
0080 EE;
0081 --PROCEDURE PRINT_NODE--procedure PRINT_NODE ( T : TREE; INDENT : NATURAL := 0 ) is
0082 --PROCEDURE PRINT_SUB_TREE--procedure PRINT_SUB_TREE ( T : TREE; IND : NATURAL is
0083 A_SIZ:= ATTR_NBR:= N_SPEC( T.TY ).NS_SIZE; N_SIZ:= ATTR_NBR:= A_SIZ; A_SUB:= INTEGER:= N_SPEC( T.TY ).NS_FIRST; TR:= TREE;
0084 SQ:= SEQ_TYPE;
0085 -- procedure PRINT_SUB_TREE ( T : TREE ) is begin PRINT_TREE( T ); if T.PT = P and then ( T.TY = DN_SYMBOL_REP and T.PG > 0 ) t
0086 hen--PUT( PRINT_NAME( T ) ); end if; end PRINT_SUB_TREE; begin if T.TY = DN_HASH then TR := DABS( 0, T ); N_SIZ :=
0087 Y DN_HASH then PUT ( ' ' ); NB_CARS := NB_CARS + L_PINT( T.PG ); PAGE OU BLOC DU-POINTEUR; PUT ( 'L' ); NB_
0088 CARS := NB_CARS + L_PINT( T.LN ) + 5; LIGNE-DU POINTEUR ET AJOUTER LES TAILLES; PUT ( ' ' ); if T.PG = 0-then--CAS INHAB
0089 ITUEL AU RELOC; return NB_CARS; ARRETER ICI; end if; if T.TY = DN_TXTREP then CAS PARTICULIER-D'UN TXTREP;
0090 PUT ( ' ' ); if T.PG > PAGE_MAN.HIGH_VPG then--HORS BORNES (1-)?PUT ( "??XT??" ); NB_CARS := NB_CARS + 7; TAILLE N
0091 OM PLUS-UN ET SIX -return NB_CARS; end if; declare NAM:= constant STRING:= PRINT_NAME( T ); begin-PUT( NAM ); NB_CA
0092 RS := NB_CARS + 1 + NAM'LENGTH; end; end if; return NB_CARS; end; when L => null; end case; return 0; end-L_PINT_TR
0093 EE;
0094 --PROCEDURE PRINT_NODE--procedure PRINT_NODE ( T : TREE; INDENT : NATURAL := 0 ) is
0095 --PROCEDURE PRINT_SUB_TREE--procedure PRINT_SUB_TREE ( T : TREE; IND : NATURAL is
0096 A_SIZ:= ATTR_NBR:= N_SPEC( T.TY ).NS_SIZE; N_SIZ:= ATTR_NBR:= A_SIZ; A_SUB:= INTEGER:= N_SPEC( T.TY ).NS_FIRST; TR:= TREE;
0097 SQ:= SEQ_TYPE;
0098 -- procedure PRINT_SUB_TREE ( T : TREE ) is begin PRINT_TREE( T ); if T.PT = P and then ( T.TY = DN_SYMBOL_REP
```